实验报告：基于自定义ResNet的表情分类

项目人员

组长： 王立宇
组员： 陈嘉楠、白博文

项目介绍

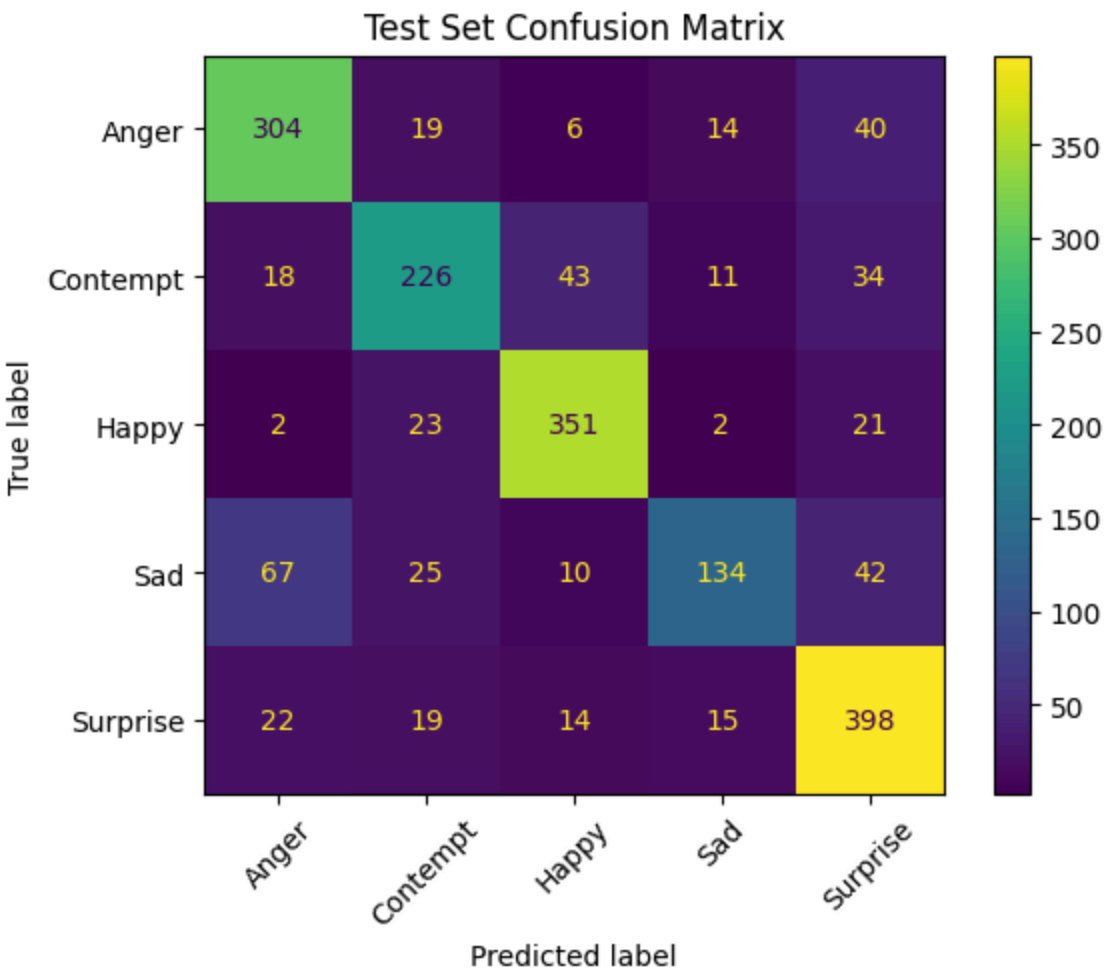
本项目实现了一个基于自定义ResNet网络结构的表情分类任务。使用PaddlePaddle深度学习框架，构建了一个简单的ResNet模型，并在公开数据集AffectNet上进行了训练和评估。

本项目中将表情分为5类。支持用户使用训练好的模型对新图片进行预测。

实验结果

测试集评估

- 最终在测试集上的准确率为：**75.97%**
- 测试集混淆矩阵如下所示，模型对各类别的区分能力较好。



预测示例

- 对 `predict` 文件夹中的图片进行预测，输出如下格式：

```
predict\surprise.jpg
预测结果: Surprise (置信度: 99.90%)
各类别概率:
Anger: 0.07%
Contempt: 0.01%
Happy: 0.00%
Sad: 0.02%
Surprise: 99.90%
```

项目结构

文件结构

```
Mode       Length Name
----       ------ ----
d-----            dataset
d-----            predict
-a----      16666 main.ipynb
-a----   85232548 resnet34_emotion.pdparams
-a----  170344878 resnet34_optimizer.pdopt
```

- `dataset`：数据集（AffectNet），分为 `train`，`val`，`test` 三个文件夹，分别储存训练集、验证集、测试集，一共分有5类：`[Anger,Contempt,Happy,Sad,Surprise]`
- `predict`：预测文件夹，可以在其中放入自己的图片进行预测
- `main.ipynb`：源代码
- `resnet34_emotion.pdparams` & `resnet34_optimizer.pdopt`：保存的模型参数和优化器参数，在加载后可用于预测

源代码结构

codeblock1:导入模块

- 导入相应模块
- 为了方便观察输出，忽略部分warning
- 设定随机性种子并使用确定性算法

codeblock2:划分数据集

- 处理原始数据，划分数据集

codeblock3:数据集加载

- 数据集增强：水平翻转、颜色抖动。
- 数据集加载。

codeblock4:定义模型

- 定义基本残差块（BasicBlock）和自定义ResNet模型（MySimpleResNet）
- 配置网络深度
- 定义类别数

codeblock5:训练并保存

- 构建模型实例，指定网络结构
- 配置损失函数与优化器
- 实现训练与验证循环，记录损失和准确率
- 保存模型参数和优化器状态

codeblock6:绘制

- 绘制训练损失、验证损失和验证准确率曲线
- 在测试集上评估模型性能，输出混淆矩阵

codeblock7:预测

- 加载保存的模型参数
- 对`predict`文件夹中的图片进行预测，输出类别及置信度

数据集处理和分析

使用了AffectNet数据集进行表情分类任务。该数据集包含多种情感标签，彩色图像，尺寸为96*96。

数据集规模：

共16,604张图片，分为5类：`[Anger,Contempt,Happy,Sad,Surprise]`，每类图片平均分布。

- 训练集：11,219张
- 验证集：3,525张
- 测试集：1,860张

数据集处理

原始数据集结构为如下格式：

```
├─test
│   ├─images
│   └─labels
├─train
│   ├─images
│   └─labels
└─valid
    ├─images
    └─labels
```

图片的分类在标签中，为了方便训练，读取标签文件，找到图片文件并将其分类到对应的文件夹中。最终数据集结构如下：

```
├─test
│   ├─Anger
│   ├─Contempt
│   ├─Happy
│   ├─Sad
│   └─Surprise
├─train
│   ├─Anger
│   ├─Contempt
│   ├─Happy
│   ├─Sad
│   └─Surprise
└─val
    ├─Anger
    ├─Contempt
    ├─Happy
    ├─Sad
    └─Surprise
```

数据集分析

- 数据较少（训练集每个分类2000张左右），采用数据增强。
- 彩色图片，输入层设置为3通道。
- 尺寸适中（96*96），可以直接输入到网络中。
- 图片干扰信息较少，每张图片均为单一表情，且背景较为简单。

实验过程与优化设计

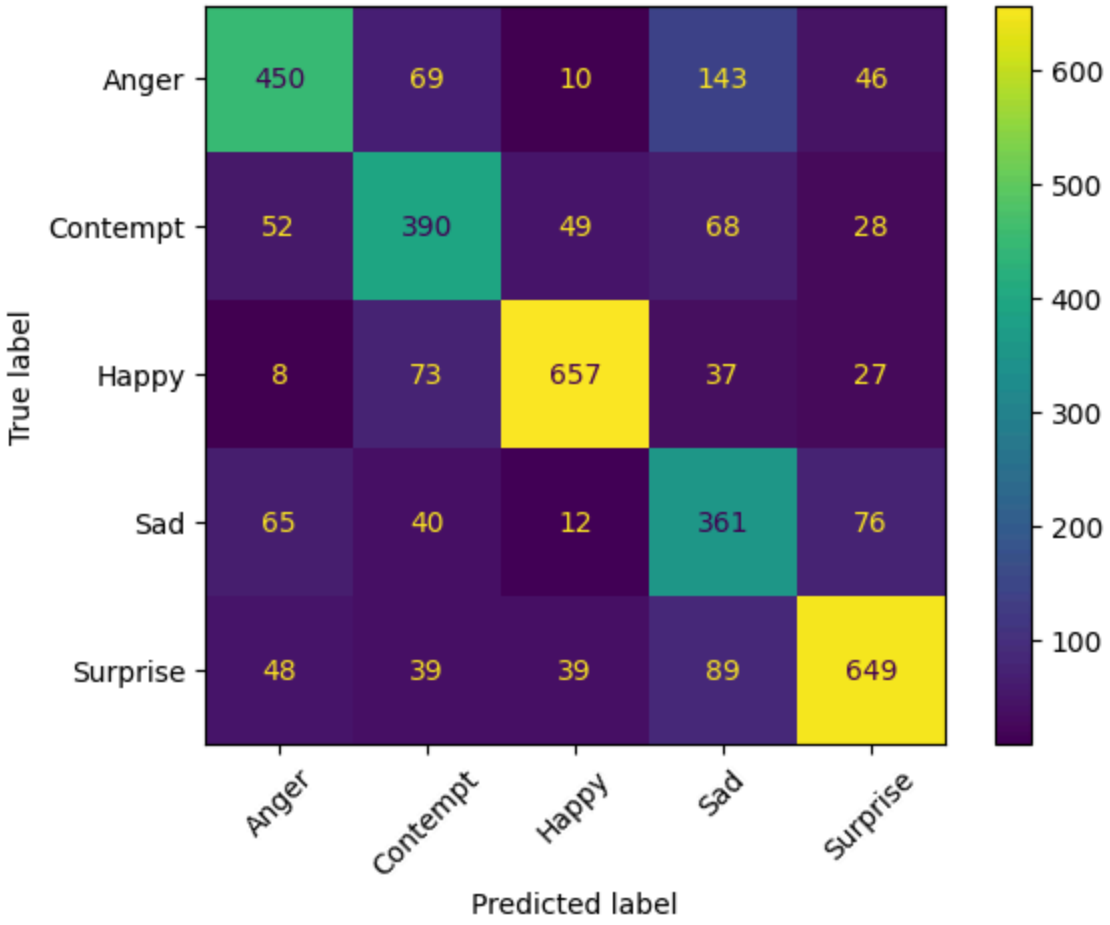
最刚开始使用的是基础的CNN，经过多次实验后在实验中发生了梯度消失的问题，导致模型无法收敛。经过查阅资料，决定优化为ResNet结构来解决这个问题。

原CNN结构

```
    def __init__(self, num_classes=5):
        super(EmotionCNN, self).__init__()
        self.conv1 = nn.Conv2D(3, 64, 3, padding=1)
        self.pool1 = nn.MaxPool2D(2)
        self.conv2 = nn.Conv2D(64, 128, 3, padding=1)
        self.pool2 = nn.MaxPool2D(2)
        self.conv3 = nn.Conv2D(128, 256, 5, padding=2)
        self.pool3 = nn.MaxPool2D(3)
        # self.conv4 = nn.Conv2D(256, 512, 3, padding=1)
        # self.pool4 = nn.MaxPool2D(2)
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(256*8*8, 256)  # 调整全连接层输入维度
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(256, num_classes)
```

预测结果：

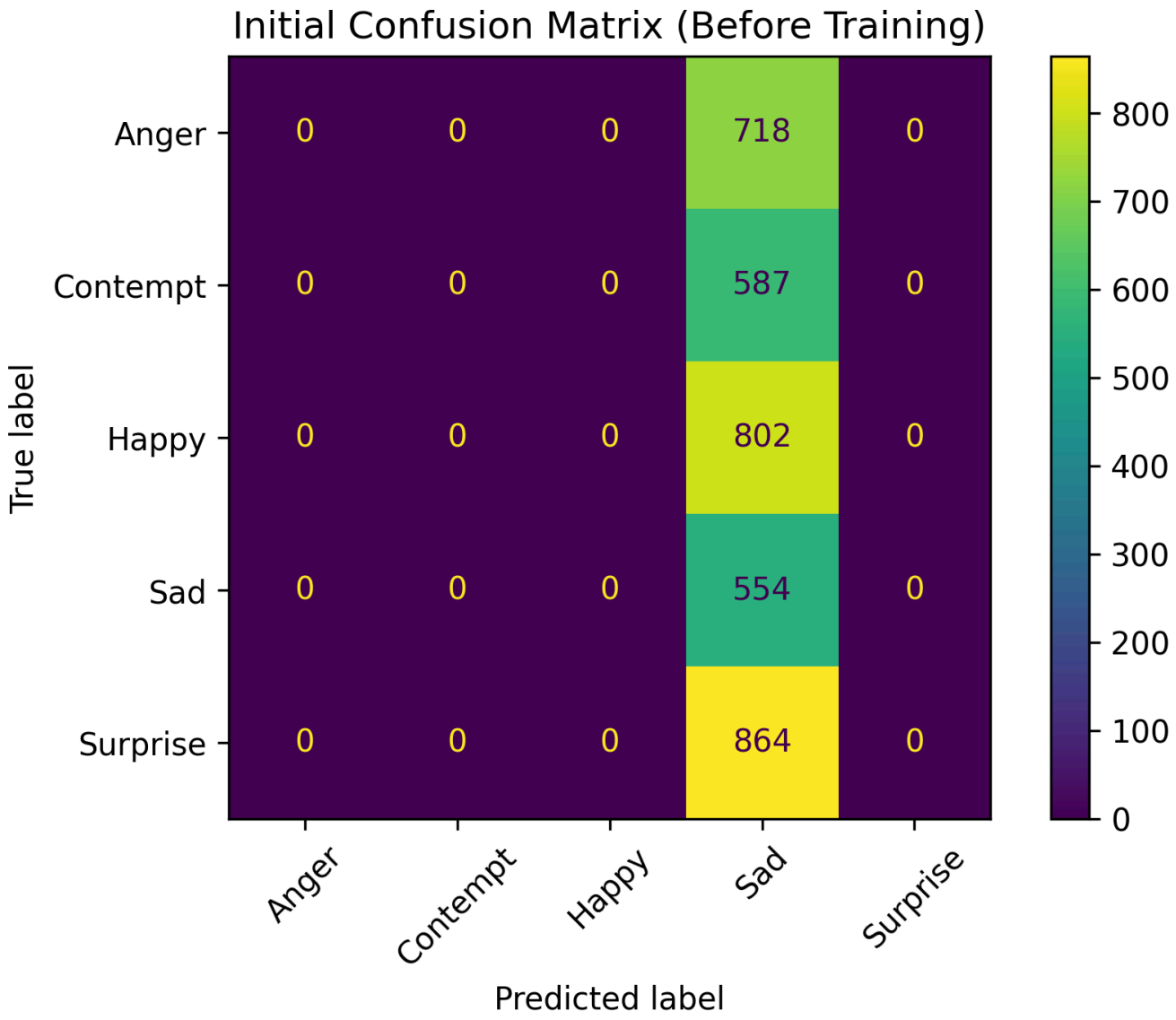
测试集准确率： 70.93%

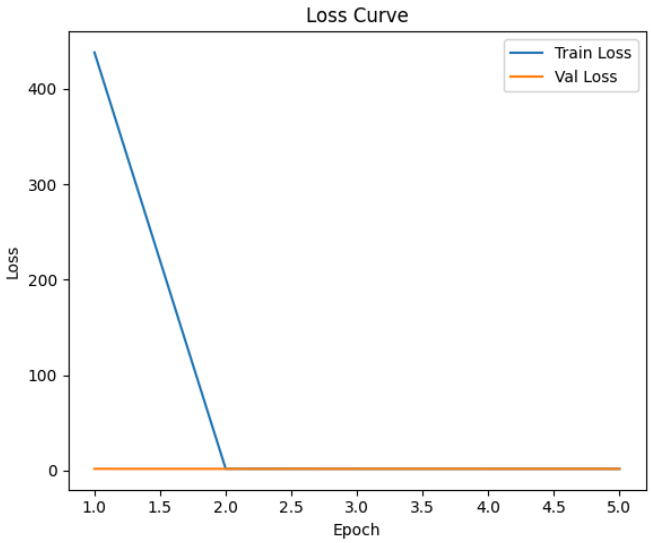


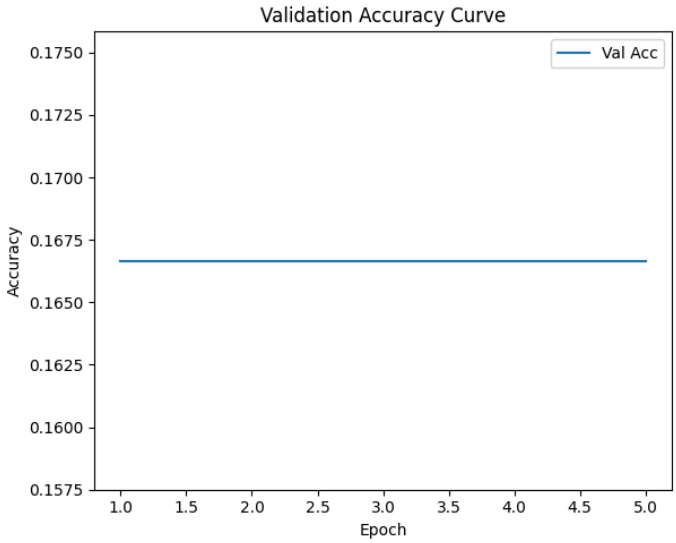
改进CNN神经网络：增加卷积/池化层数

出现问题：梯度消失，测试集的所有结果都为同一类。

```
Epoch 1, Train Loss: 438.1232, Val Loss: 1.8244, Val Acc: 0.1667
Epoch 2, Train Loss: 1.8670, Val Loss: 1.7936, Val Acc: 0.1667
Epoch 3, Train Loss: 1.8129, Val Loss: 1.7922, Val Acc: 0.1667
Epoch 4, Train Loss: 1.8084, Val Loss: 1.7920, Val Acc: 0.1667
Epoch 5, Train Loss: 1.8076, Val Loss: 1.7919, Val Acc: 0.1667
Test Loss: 1.7919, Test Accuracy: 0.1667
```


Initial Confusion Matrix (Before Training)
True label
Predicted label
Anger
Contempt
Happy
Sad
Surprise
0 0 0 718 0
0 0 0 587 0
0 0 0 802 0
0 0 0 554 0
0 0 0 864 0
800
700
600
500
400
300
200
100
0


Loss Curve
Train Loss
Val Loss
Loss
Epoch
400
300
200
100
0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0


Validation Accuracy Curve
Val Acc
Accuracy
Epoch
0.1750
0.1725
0.1700
0.1675
0.1650
0.1625
0.1600
0.1575
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

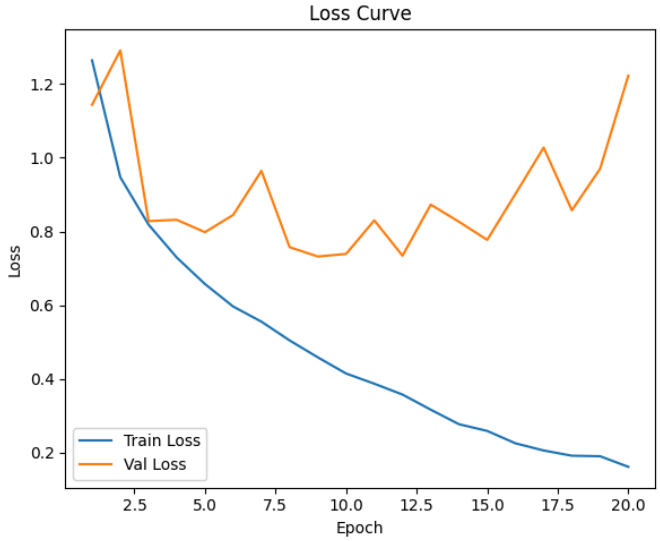
进一步改进：引入残差块，自定义ResNet，允许梯度直接从深层反向传播到浅层，避免梯度消失

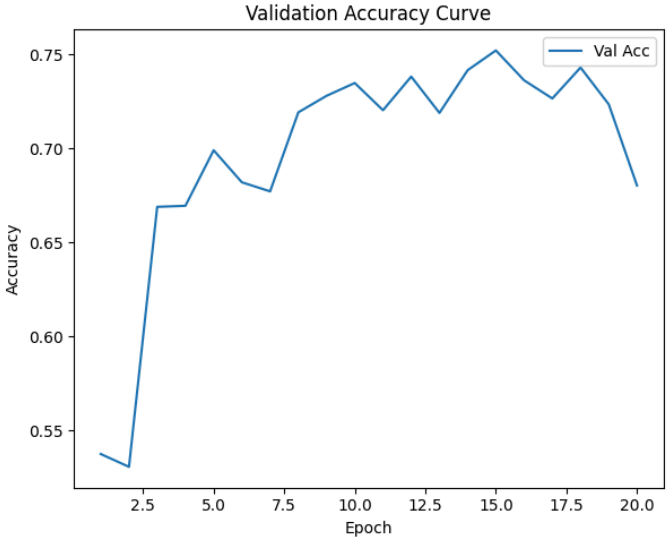
```
#  2.实现基本残差块
class BasicBlock(nn.Layer):
    expansion = 1 #输入输出通道数在块内不扩展

    def __init__(self, in_channels, out_channels, stride=1, downsample=None):
        super(BasicBlock, self).__init__()
        # 第一个卷积层
        self.conv1 = nn.Conv2D(in_channels, out_channels,
         kernel_size=3, stride=stride, padding=1, bias_attr=False)
        self.bn1 = nn.BatchNorm2D(out_channels)
        self.relu = nn.ReLU()
        # 第二个卷积层
        self.conv2 = nn.Conv2D(out_channels, out_channels,
         kernel_size=3, stride=1, padding=1, bias_attr=False)
        self.bn2 = nn.BatchNorm2D(out_channels)

        self.downsample = downsample # 用于处理快捷连接维度不匹配的情况
        self.stride = stride

    def forward(self, x):
        identity = x # 保存输入，即快捷连接的起点

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
        out = self.conv2(out)
        out = self.bn2(out)

        out += identity # 核心：将卷积输出与原始输入（或经过下采样）相加
        out = self.relu(out)

        return out
```

```python
# 3. 定义 ResNetMyself 模型

class MySimpleResNet(nn.Layer):
    def __init__(self, block, layers_cfg, num_classes=5):
        # layers_cfg 是一个列表，表示每个阶段的块数量，例如 [2, 2, 2, 2]
        super(MySimpleResNet, self).__init__()
        self.in_channels = 64 # 初始输入通道数

        # 初始卷积层 (Stem)
        self.conv1 = nn.Conv2D(3, self.in_channels, kernel_size=7,
         stride=2, padding=3, bias_attr=False)
        self.bn1 = nn.BatchNorm2D(self.in_channels)
        self.relu = nn.ReLU()
        self.maxpool = nn.MaxPool2D(kernel_size=3, stride=2, padding=1)

        # 构建残差层
        self.layer1 = self._make_layer(block, 64, layers_cfg[0])
        self.layer2 = self._make_layer(block, 128, layers_cfg[1], stride=2) # stride=2进行下采样
        self.layer3 = self._make_layer(block, 256, layers_cfg[2], stride=2)
        # self.layer4 = self._make_layer(block, 512, layers_cfg[3], stride=2)

        self.avgpool = nn.AdaptiveAvgPool2D((1, 1)) # 全局平均池化

        self.fc = nn.Linear(current_fc_in_features, num_classes)

    def _make_layer(self, block, out_channels, blocks_num, stride=1):
        downsample = None
        # 当stride不为1（需要下采样使尺寸匹配）或输入输出通道数不一致时，需要定义downsample层
        if stride != 1 or self.in_channels != out_channels * block.expansion:
            downsample = nn.Sequential(
                nn.Conv2D(self.in_channels, out_channels * block.expansion,
                kernel_size=1, stride=stride, bias_attr=False),
                nn.BatchNorm2D(out_channels * block.expansion),
            )

        layers = []
        layers.append(block(self.in_channels, out_channels, stride, downsample))
        self.in_channels = out_channels * block.expansion # 更新下一层的输入通道数
        for _ in range(1, blocks_num): # 堆叠其余的块
```

```
            layers.append(block(self.in_channels, out_channels))

        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.relu(x)
        x = self.maxpool(x)

        x = self.layer1(x)
        x = self.layer2(x)
        x = self.layer3(x)
        # if hasattr(self, 'layer4'):
        #     x = self.layer4(x)

        x = self.avgpool(x)
        x = paddle.flatten(x, 1)
        x = self.fc(x)
        return x
```

出现问题：过拟合，Train Loss持续下降，逐渐趋近于0，Val Loss却在后期保持不变。





对于过拟合问题的解决方法

动态降低学习率

出现过拟合的原因之一是学习率过大，最后阶段模型梯度下降时的步长过大，导致模型无法收敛。

设置每4轮学习率等比下降

```
# 设定 ResNetMyself 的学习率
scheduler = paddle.optimizer.lr.StepDecay(learning_rate=0.001, step_size=4, gamma=0.4)
optimizer = Adam(parameters=model.parameters(), learning_rate=scheduler, weight_decay=1e-4)
```

根据原有的混淆矩阵，调整权重

```
# 类别 2，3，4 易混淆，增加其样本权重
class_weights = paddle.to_tensor([1.0, 1.2, 1.2, 1.1, 1.0])
criterion = nn.CrossEntropyLoss(weight=class_weights)
```

早退

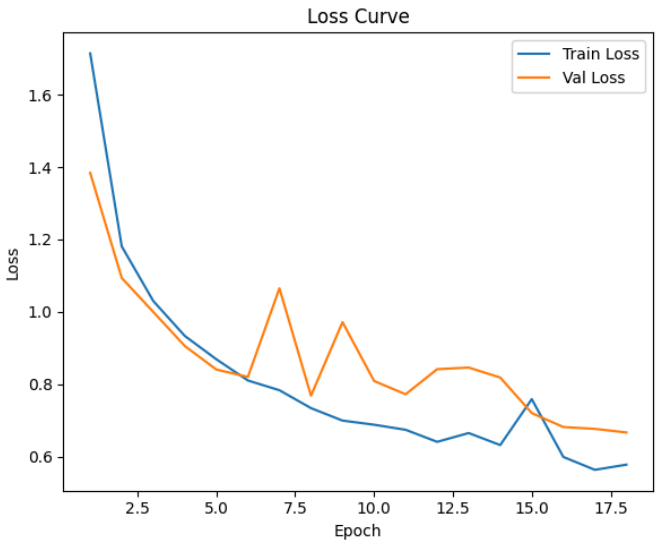
根据先前的经验，Eoch>15时，Val Loss不再下降，停止训练。因此，根据实验结果，将Eochs设置为18.

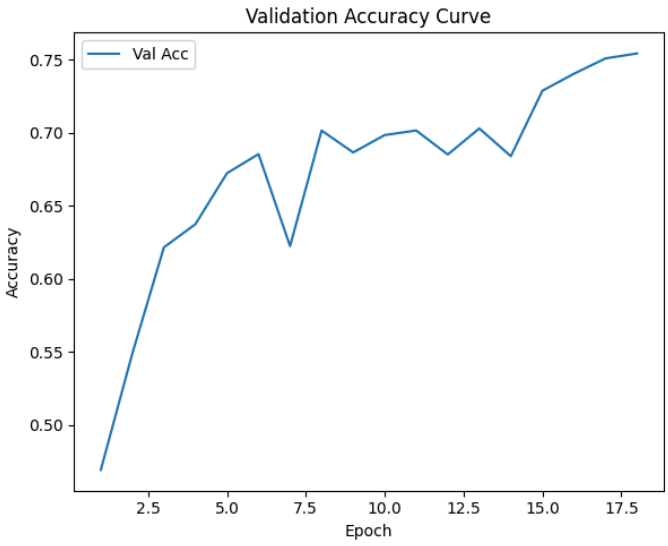
添加dropout

以50%的概率随机丢弃神经元，防止过拟合。

```
self.dropout = nn.Dropout(p=0.5)  # 添加Dropout层
```

改善之后的模型在测试集上的准确率为：**75.97%**





总结与分析

模型性能对比

- **原始CNN模型**在测试集上准确率为70.93%，存在梯度消失和过拟合问题，导致训练后期模型无法有效收敛。
- **改进后的ResNet模型**通过引入残差结构，显著缓解了梯度消失问题，最终测试准确率提升至75.97%，验证了残差连接在深层网络中的有效性。

改进措施的有效性

1. **残差块设计**：通过跨层连接保留浅层特征，确保梯度直接反向传播，解决了深层网络训练中的梯度消失问题。
2. **动态学习率与正则化**：
   - 学习率衰减（StepDecay）和权重衰减（L2正则化）有效控制了优化过程的稳定性。
   - Dropout层（丢弃率0.5）显著降低了过拟合风险，验证集损失曲线趋于平稳。
3. **类别权重调整**：针对易混淆类别（如Contempt、Sad）增加样本权重，改善了类别间分类性能的均衡性。

数据集挑战与应对

- **数据量不足**：训练集仅11,219张图片，通过水平翻转、颜色抖动等数据增强手段扩充样本多样性。
- **类别混淆**：混淆矩阵显示Contempt与Sad、Anger等类别存在误判，未来可通过针对性数据

增强（如局部遮挡、噪声添加）进一步优化。

实验结果意义

- 在AffectNet五分类任务中，75.97%的准确率表明模型具备较好的表情识别能力，但与SOTA模型（通常>85%）相比仍有提升空间。
- 模型参数量适中（ResNet34简化版），在保证推理速度的同时平衡了性能需求。

未来改进方向

1. **模型结构优化**：尝试更深的ResNet变体（如ResNet50）或引入注意力机制（如SE模块），提升特征提取能力。
2. **数据增强扩展**：引入混合样本（Mixup）、随机擦除（Random Erasing）等增强策略，增强模型泛化性。
3. **损失函数改进**：采用Focal Loss缓解类别不平衡问题，或使用标签平滑技术提升分类置信度校准。
4. **部署优化**：结合模型剪枝或量化技术，降低计算资源消耗，适配边缘设备部署需求。

项目收获

- 通过实践深入理解了残差网络的设计原理及其在图像分类任务中的优势。
- 掌握了过拟合控制、学习率调度、类别权重调整等关键调参技巧。
- 验证了PaddlePaddle框架在模型开发与部署中的高效性，为后续研究奠定技术基础。

局限性

- 模型对部分表情（如Contempt）的区分能力较弱，需针对性优化数据或引入预训练模型特征。
- 当前实验仅覆盖5类表情，未来可扩展至更细粒度的情感分类任务（如28类AffectNet全类别）。